VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

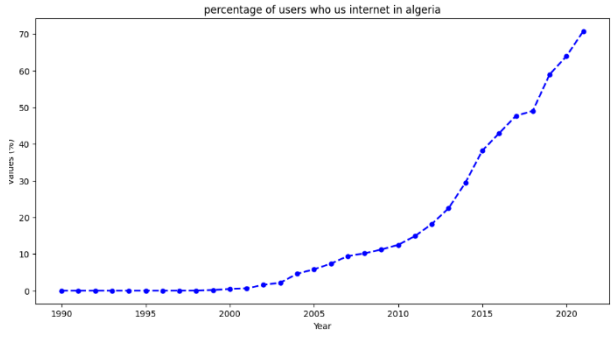
Name of the Experiment: _____

Experiment No: ____ Date: _____

WEEK 5- To implement simple and multiple linear regression models using Python and analyze their performance on training data from a CSV file.

LINEAR REGRESSION

```
import numpy as np
import pandas as pfd
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn.model_selection import train_test_split
from pandas.core.common import random_state
from sklearn.linear_model import LinearRegression
import scipy.stats as stats


import kagglehub
path = kagglehub.dataset_download("fundal/algeria-five-indicators")
print("Path to dataset files:", path)


import os
import pandas as pd
print(os.listdir(path))
dataset_file = os.path.join(path, 'Four_Indicators.csv')
df = pd.read_csv(dataset_file)
display(df.head())


df.describe()
df


plt.figure(figsize=(12,6))
plt.title("percentage of users who us internet in algeria")
plt.xlabel('Year')
plt.ylabel('values (%)')
df.Users_Percentage.plot(kind='line',color='blue',marker='o',linestyle='dashed',linewidth=2,
markersize=5)
```

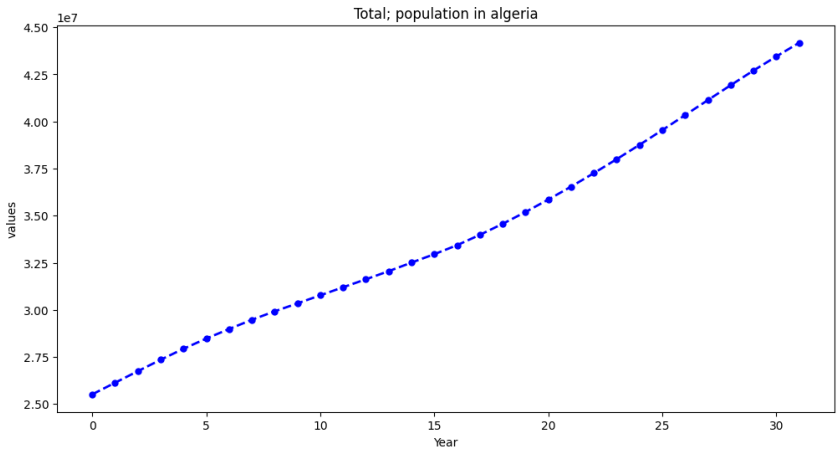


```
indic_df=pd.read_csv(os.path.join(path,'Four_Indicators.csv'))
indic_df
```

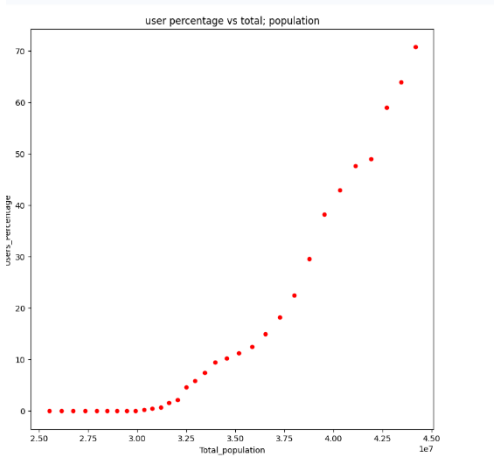
```
from IPython.core.interactiveshell import dis
indic_df.dropna(how='all', axis=1,inplace=True)
display(indic_df)
```

	Year	Users_Percentage	Total_population	Fixed_telephone_subscriptions	Mobile_phone_subscriptions	GDP_per_capita_current_US_Dollar
0	1990	0.000000	25518074	812000	470	2431.551360
1	1991	0.000000	26133905	883120	4781	1749.286087
2	1992	0.000000	26748303	962247	4781	1794.623507
3	1993	0.000000	27354327	1068094	4781	1825.875097
4	1994	0.000361	27937006	1122409	1348	1522.825203
5	1995	0.001769	28478022	1176316	4691	1466.544680
6	1996	0.001739	28984634	1278142	11700	1619.532412
7	1997	0.010268	29476031	1400343	17400	1634.467410
8	1998	0.020239	29924668	1477000	18000	1610.302978
9	1999	0.199524	30346083	1600000	72000	1602.864915
10	2000	0.491706	30774621	1761327	86000	1780.376063
11	2001	0.646114	31200985	1880000	100000	1754.582361
12	2002	1.591641	31624696	1950000	450244	1794.811114
13	2003	2.195360	32055883	2079464	1446927	2117.048229
14	2004	4.634475	32510186	2486720	4882414	2624.795232
15	2005	5.843942	32956690	2572000	13661355	3131.328190
16	2006	7.375985	33435080	2841297	20997954	3500.134594
17	2007	9.451191	33983827	3068409	27562721	3971.803488
18	2008	10.180000	34569592	3069140	27031472	4946.564017
19	2009	11.230000	35196037	2576165	32729824	3898.478788
20	2010	12.500000	35856344	2922731	32780165	4495.921476
21	2011	14.900000	36543541	3059336	35615926	5473.281826

```
plt.figure(figsize=(12,6))
plt.title("Total; population in algeria")
plt.xlabel('Year')
plt.ylabel('values')
indic_df.Total_population.plot(kind='line',color='blue',marker='o',linestyle='dashed',linewidth=2, markersize=5)
```

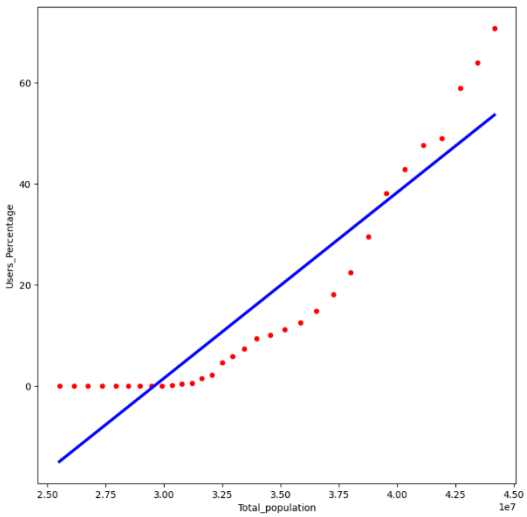


```
indic_df.plot(kind='scatter',x='Total_population',y='Users_Percentage',figsize=(9,9),color='red')
plt.title("user percentage vs total; population")
plt.show()
```




```
regression_model=linear_model.LinearRegression()
regression_model.fit(indic_df[['Total_population']],y=indic_df["Users_Percentage"])
print("y-intercept")
print(regression_model.intercept_)
print("------------")
print("model coefficient")
print(regression_model.coef_)
print("--------")
print("score")
regression_model.score(indic_df[['Total_population']],indic_df["Users_Percentage"])
train_prediction=repr(regression_model.predict(indic_df[['Total_population']]))
print(train_prediction)
residuals=indic_df["Users_Percentage"]-regression_model.predict(indic_df[['Total_population']])
indic_df.plot(kind='scatter',x='Total_population',y='Users_Percentage',figsize=(9,9),color='red')
plt.plot(indic_df[['Total_population']],regression_model.predict(indic_df[['Total_population']]),color='bl
ue',linewidth=3)
```

```
y-intercept
-108.68753628401305
------------
model coefficient
[3.67435754e-06]
--------
score
array([-14.92500857, -12.66222529, -10.40470736,  -8.1779585 ,
        -6.03698752,  -4.0491013 ,  -2.18762768,  -0.3820594 ,
         1.26639334,   2.81482273,   4.38942456,   5.95603834,
         7.51290405,   9.09723925,  10.76651091,  12.40712625,
        14.16490216,  16.18119484,  18.33350488,  20.63528779,
        23.0614918 ,  25.58649928,  28.22109448,  30.94035055,
        33.73117942,  36.60814994,  39.53358156,  42.46284186,
        45.36727819,  48.22725481,  50.96942049,  53.6381174 ])
```



Conclusion:

The graph shows that urban population increases as total population increases, indicating a positive relationship between the two. However, the data follows a curved pattern while the fitted line is linear, so the model does not represent the trend accurately. This suggests that a non-linear model would better capture the relationship than a simple linear regression.

MULTIPLE REGRERSSION

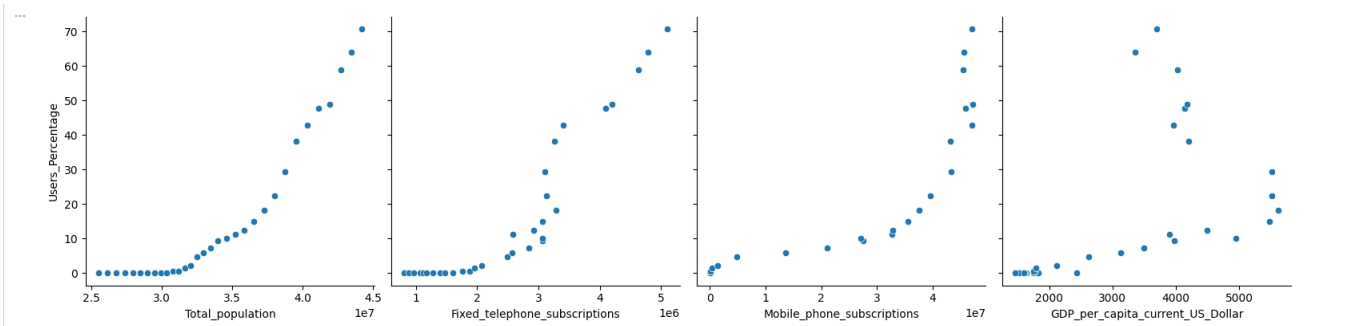
```
from sklearn import metrics
from sklearn.metrics import mean_squared_error,mean_absolute_error
from sklearn.model_selection import train_test_split, cross_val_score
```

indic_df.describe()

	Year	Users_Percentage	Total_population	Fixed_telephone_subscriptions	Mobile_phone_subscriptions	GDP_per_capita_current_US_Dollar
count	32.000000	32.000000	3.200000e+01	3.200000e+01	3.200000e+01	32.000000
mean	2005.500000	16.345217	3.402847e+07	2.533123e+06	1.997168e+07	3154.423049
std	9.380832	21.651908	5.444793e+06	1.211562e+06	1.999630e+07	1413.127958
min	1990.000000	0.000000	2.551807e+07	8.120000e+05	4.700000e+02	1466.544680
25%	1997.750000	0.017746	2.981251e+07	1.457836e+06	1.785000e+04	1773.927637
50%	2005.500000	6.609964	3.319588e+07	2.574082e+06	1.732965e+07	3242.740677
75%	2013.250000	24.250000	3.819051e+07	3.166520e+06	4.044469e+07	4144.149662
max	2021.000000	70.770000	4.417797e+07	5.097059e+06	4.715426e+07	5610.733282

```
x=indic_df[["Total_population","Fixed_telephone_subscriptions","Mobile_phone_subscriptions","GDP_per_capita_current_US_Dollar"]]
y=indic_df['Users_Percentage']
```

```
import seaborn as sns
sns.pairplot(indic_df,
x_vars=["Total_population","Fixed_telephone_subscriptions","Mobile_phone_subscriptions","GDP_per_capita_current_US_Dollar"], y_vars="Users_Percentage", height=4, aspect=1, kind='scatter')
plt.show()
```



VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

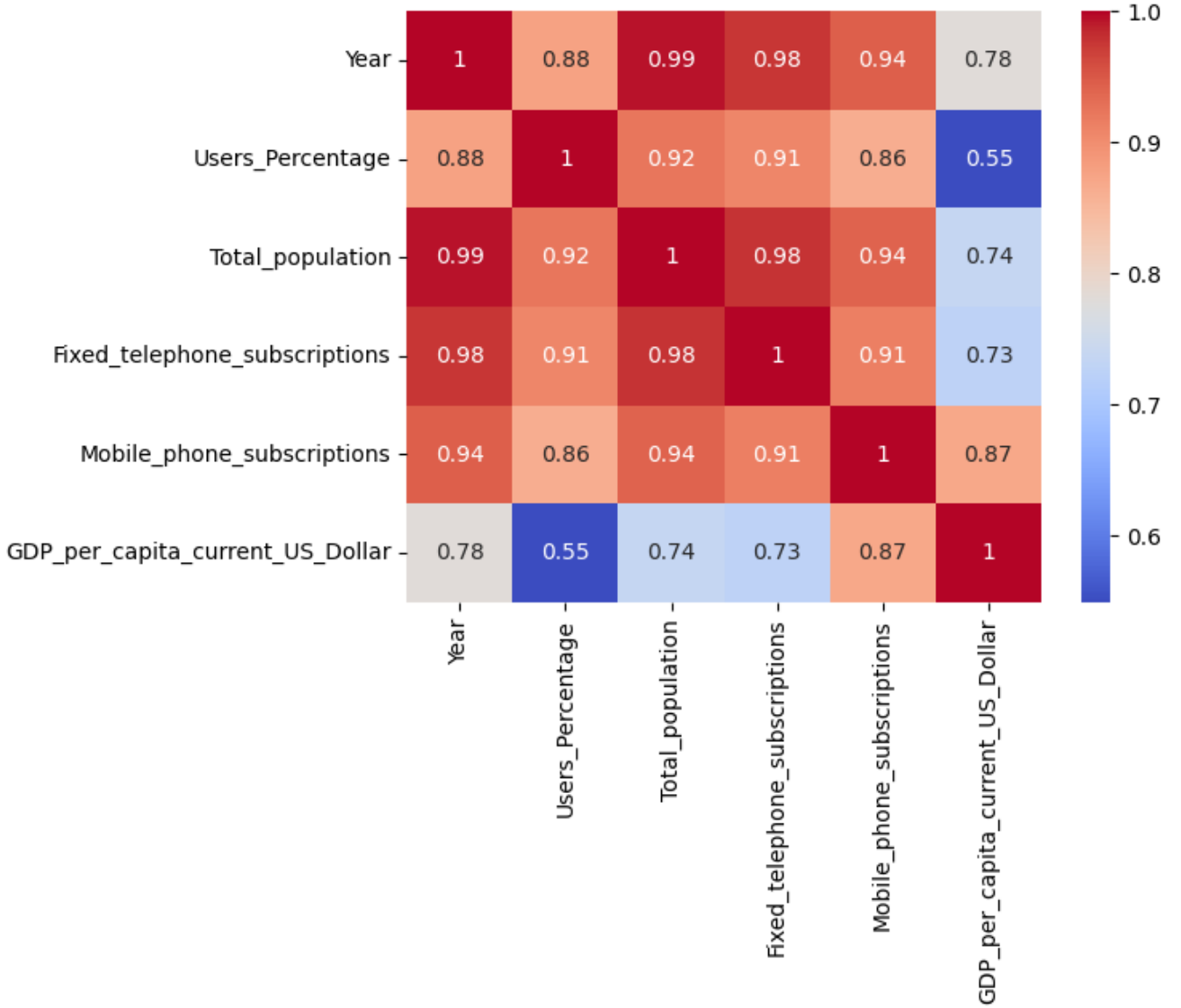
Name of the Experiment: _____

Experiment No: ____ **Date:** _____

indic_df.corr()

	Year	Users_Percentage	Total_population	Fixed_telephone_subscriptions	Mobile_phone_subscriptions	GDP_per_capita_current_US_Dollar
Year	1.000000	0.877656	0.993383	0.975367	0.943780	0.780569
Users_Percentage	0.877656	1.000000	0.923989	0.908183	0.864147	0.549066
Total_population	0.993383	0.923989	1.000000	0.977937	0.941829	0.737172
Fixed_telephone_subscriptions	0.975367	0.908183	0.977937	1.000000	0.913755	0.725376
Mobile_phone_subscriptions	0.943780	0.864147	0.941829	0.913755	1.000000	0.869880
GDP_per_capita_current_US_Dollar	0.780569	0.549066	0.737172	0.725376	0.869880	1.000000

sns.heatmap(indic_df.corr(),annot=True,cmap='coolwarm')
plt.show()



x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2, random_state=100)

```
y_train.shape
```

```
(7, 4)
```

```
y_test.shape
```

```
(7,)
```

```
reg_model=linear_model.LinearRegression()
reg_model=LinearRegression().fit(x_train,y_train)
print('intercept',reg_model.intercept_)
print('slope',reg_model.coef_)
```

```
intercept -18.723242354209347
slope [ 1.10380338e-06  3.38175345e-06  1.13095535e-06 -1.07438261e-02]
```

```
y_pred=reg_model.predict(x_test)
y_pred=reg_model.predict(x_test)
reg_model_diff=pd.DataFrame({'Actual value':y_test,'Predicted value':y_pred,'Difference':y_test-y_pred})
reg_model_diff
```

	Actual value	Predicted value	Difference
13	2.195360	2.583596	-0.388236
28	49.038468	50.270779	-1.232311
1	0.000000	-5.678674	5.678674
26	42.945527	47.896051	-4.950525
5	0.001769	0.937910	-0.936141
18	10.180000	7.240229	2.939771
29	58.977575	52.254510	6.723064

```
y_pred = reg_model.predict(x_test)

print('R-squared score on test set:', reg_model.score(x_test, y_test))
print('Mean Absolute Error (MAE) on test set:', metrics.mean_absolute_error(y_test, y_pred))
print('Mean Squared Error (MSE) on test set:', metrics.mean_squared_error(y_test, y_pred))
```

```
R-squared score on test set: 0.9718867976426132
Mean Absolute Error (MAE) on test set: 3.2641029965582407
Mean Squared Error (MSE) on test set: 16.163221676848206
```

VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

Name of the Experiment: _____

Experiment No: ____ Date: _____

WEEK 6: Write a program to implement the k-nearest neighbour algorithm for classifying iris data sets and printing both correct and incorrect predictions.

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
import numpy as np



dataset=load_iris()
x_train,x_test,y_train,y_test=train_test_split(dataset["data"],dataset["target"],random_state=50)


kn=KNeighborsClassifier(n_neighbors=1)
kn.fit(x_train,y_train)



kn=KNeighborsClassifier(n_neighbors=2)
kn.fit(x_train,y_train)


kn=KNeighborsClassifier(n_neighbors=5, weights='uniform', algorithm='auto', leaf_size=30, p=3,
metric='minkowski', metric_params=None, n_jobs=None)


kn.fit(x_train,y_train)
for i in range(len(x_test)):
  x=x_test[i]
  x_new=np.array([x])
  prediction=kn.predict(x_new)
  print('Target=',y_test[i],dataset["target_names"][y_test[i]],"predicted=",prediction,dataset)
print(kn.score(x_test,y_test))
```

```
kn=KNeighborsClassifier(n_neighbors=10, weights='uniform', algorithm='ball_tree',
leaf_size=30, p=3, metric='minkowski', metric_params=None, n_jobs=None)



kn.fit(x_train,y_train)
for i in range(len(x_test)):
  x=x_test[i]
  x_new=np.array([x])
  prediction=kn.predict(x_new)
  print('Target=',y_test[i],dataset["target_names"][y_test[i]],"predicted=",prediction,datas
et)
print(kn.score(x_test,y_test))
```

```
Streaming output truncated to the last 5000 lines.
       [5.9, 3. , 5.1, 1.8]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
...    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]), 'frame': None, 'target_names': array(['setosa', 'vers
Target= 2 virginica predicted= [2] {'data': array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4],
       [5.4, 3.9, 1.3, 0.4],
       [5.1, 3.5, 1.4, 0.3],
```

VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

Name of the Experiment: _____

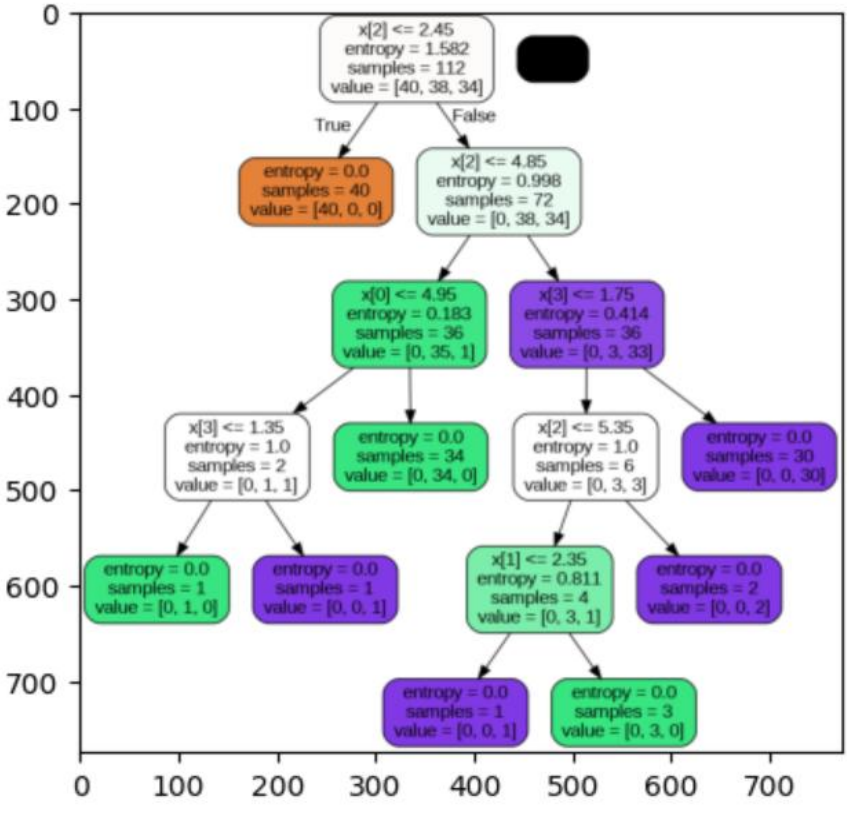
Experiment No: ____ Date: _____

WEEK 7: Write a program to implement a decision tree using the ID3 algorithm and apply the knowledge to classify a new sample using the decision tree.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier,export_graphviz,ExtraTreeClassifier
iris=load_iris()
```

```
iris.data[:5]
iris.target[:5]

dt=DecisionTreeClassifier(criterion='entropy')
from sklearn.model_selection import train_test_split

trainx,testx,trainy,testy=train_test_split(iris.data,iris.target)
dt.fit(trainx,trainy)

DecisionTreeClassifier(class_weight=None,criterion='entropy',
max_depth=None,max_features=None,max_leaf_nodes=None,min_impurity_decrease=0
.0, min_samples_leaf=1, min_samples_split=2, min_weight_fraction_leaf=0.0,
random_state=None, splitter='best')

export_graphviz(dt,'dt.tree')

from sklearn import tree
import pydotplus

dot_data_string = tree.export_graphviz(dt, filled=True, rounded=True)
graph = pydotplus.graph_from_dot_data(dot_data_string)
graph.write_png("tree.png")
import matplotlib.pyplot as plt
plt.imshow(plt.imread("tree.png"))
```



VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

Name of the Experiment: _____

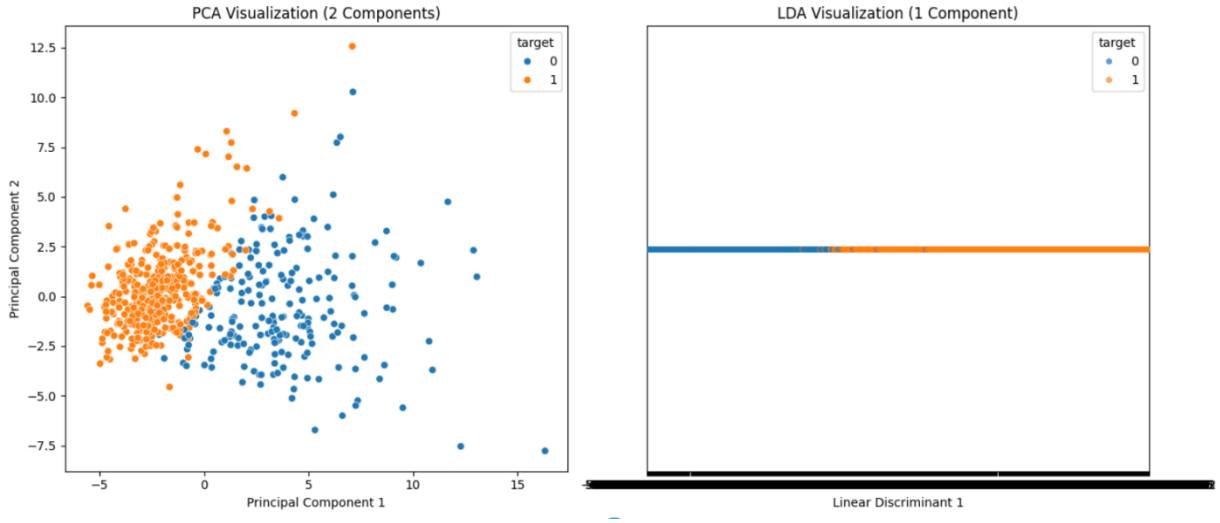
Experiment No: ____ Date: _____

WEEK 8: To implement different feature selection and extraction techniques (LDA & PCA) using Python

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from time import time
from sklearn.datasets import load_wine, load_breast_cancer
from sklearn.preprocessing import StandardScaler


from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.decomposition import PCA



cancer = load_breast_cancer()
data = cancer.data
columns = cancer.feature_names
X = pd.DataFrame(data, columns=columns)
y = pd.Series(cancer.target, name='target')
df = pd.concat([X, y], axis=1)
print(df.sample(10))


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)




def apply_model(X, y):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, random_state=1, test_size=0.3
    )
    model = LogisticRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    print("Accuracy:", accuracy_score(y_test, y_pred))




# Re-compute PCA for visualization using breast cancer data
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
df_pca = pd.concat([pd.DataFrame(X_pca, columns=['PC1', 'PC2']), y], axis=1)

# Prepare LDA data for visualization
# X_lda is already a 1-D array from previous step, we'll give it a column name for plotting
df_lda_viz = pd.concat([pd.DataFrame(X_lda, columns=['LD1']), y], axis=1)

# Create the visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# PCA Visualization
sns.scatterplot(x='PC1', y='PC2', hue='target', data=df_pca, ax=axes[0])
axes[0].set_title('PCA Visualization (2 Components)')
axes[0].set_xlabel('Principal Component 1')
axes[0].set_ylabel('Principal Component 2')

# LDA Visualization (1 Component)
# Using stripplot for 1D data separation by target
sns.stripplot(x='LD1', y=[0]*len(df_lda_viz), hue='target', data=df_lda_viz, ax=axes[1], jitter=0.2,
alpha=0.7)
axes[1].set_title('LDA Visualization (1 Component)')
axes[1].set_xlabel('Linear Discriminant 1')
axes[1].set_ylabel('')
axes[1].set_yticks([]) # Remove y-axis ticks for better 1D visualization

plt.tight_layout()
plt.show()
```



```
lda = LinearDiscriminantAnalysis(n_components=1)
lda.fit(X_scaled, y)
X_lda = lda.transform(X_scaled)
df_lda = pd.concat([pd.DataFrame(X_lda), y], axis=1)


print("Original shape:", X_scaled.shape)
print("Reduced shape:", X_lda.shape)
```

OUTPUT:
Original shape: (569, 30)
Reduced shape: (569, 1)

VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

Name of the Experiment: _____

Experiment No: ____ Date: _____

WEEK 9: Write a program to implement the naive Bayesian classifier for a sample training dataset stored as a CSV file. Compute the accuracy of the classifier, considering a few test data sets

```
import kagglehub
path = kagglehub.dataset_download("uciml/pima-indians-diabetes-database")
print("Path to dataset files:", path)


import pandas as pd
df = pd.read_csv(os.path.join(path, 'diabetes.csv'))
print("Dataset loaded successfully. First 5 rows:")
print(df.head())
```

```
import os
print(os.listdir(path))
import csv
import random
import math


def loadcsv(filename):
  lines=csv.reader(open(filename,"r"))
  dataset=list(lines)
  # Skip the header row
  if len(dataset) > 0:
    header = dataset.pop(0)
  for i in range(len(dataset)):
    dataset[i]=[float (x) for x in dataset[i]]
  return dataset


def splitdataset(dataset,splitratio):
  trainsize=int(len(dataset)*splitratio)
  trainset=[]
  copy=list(dataset)
  while len(trainset)<trainsize:
    index=random.randrange(len(copy))
    trainset.append(copy.pop(index))
  return [trainset,copy]


def mean (numbers):
  return sum(numbers)/float(len(numbers))


def stdev(numbers):
  avg=mean (numbers)
  variance=sum([pow(x-avg,2)for x in numbers])/float(len(numbers)-1)
  return math.sqrt(variance)


def summarize (dataset):
  summaries=[(mean(attribute),stdev(attribute))for attribute in zip(*dataset)]
  del summaries[-1]
  return summaries


def summarizebyclass(dataset):
```

```
 seperated=seperatedbyclass(dataset)
 summaries={}
 for classvalue,instances in seperated.items():
  summaries[classvalue]=summarize(instances) # Corrected 'instance' to 'instances'
 return summaries


def calculateProbability (x,mean,stdev):
 exponent=math.exp(-(math.pow(x-mean,2)/(2*math.pow(stdev,2))))
 return (1/(math.sqrt(2*math.pi)*stdev))*exponent


def calculateClassProbabilities(summaries,inputvector):
 probabilities={}
 for classvalue,classsumaries in summaries.items():
  probabilities[classvalue]=1
  for i in range(len(classsumaries)):
   mean,stdev=classsumaries[i]
   x=inputvector[i]
   probabilities[classvalue]*=calculateProbability(x,mean,stdev)
 return probabilities


def predict(summaries,inputvector):
 probabilities=calculateClassProbabilities(summaries,inputvector)
 bestlabel,bestprob=None,-1 # Corrected 'none' to 'None'
 for classvalue,prob in probabilities.items(): # Changed 'probabilities' to 'prob' to avoid shadowing
  if bestlabel is None or prob > bestprob: # Used 'prob' for comparison
   bestprob=prob
   bestlabel=classvalue
 return bestlabel


def getPedictions(summaries,testset):
 predictions=[]
 for i in range (len(testset)):
  result=predict(summaries,testset[i])
  predictions.append(result)
 return predictions
def getaccuracy(testset,predictions):
 correct=0
```

```
  for i in range(len(testset)):
    if testset[i][-1]==predictions[i]:
      correct+=1
  return(correct/float(len(testset)))*100.0


def main():
  filename=os.path.join(path, 'diabetes.csv') # Corrected: filename should be the path to the csv file
  splitratio=0.9
  dataset=loadcsv(filename)
  print("length of dataset",len(dataset))
  print("dataset splitting")
  trainset,testset=splitdataset(dataset,splitratio)
  print("num of rows :{0}".format(len(trainset)))
  print("first five rows of testing split")
  for i in range(0,5):
    print(testset[i],"\n")
    summaries=summarizebyclass(trainset)
    predictions=getPedictions(summaries,testset)
    accuracy=getaccuracy(testset,predictions)
    print("accuracy:{0}/".format(accuracy))
main()
```

```
length of dataset 768
dataset splitting
num of rows :691
first five rows of testing split
[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0, 0.0]

accuracy:64.93506493506493/
[8.0, 125.0, 96.0, 0.0, 0.0, 0.0, 0.232, 54.0, 1.0]

accuracy:64.93506493506493/
[4.0, 110.0, 92.0, 0.0, 0.0, 37.6, 0.191, 30.0, 0.0]

accuracy:64.93506493506493/
[1.0, 189.0, 60.0, 23.0, 846.0, 30.1, 0.398, 59.0, 1.0]

accuracy:64.93506493506493/
[1.0, 103.0, 30.0, 38.0, 83.0, 43.3, 0.183, 33.0, 0.0]

accuracy:64.93506493506493/
```

VNR VJIET

Name of the Laboratory:
Machine Learning Laboratory

Name of the Experiment: _____

Experiment No: ____ Date: _____

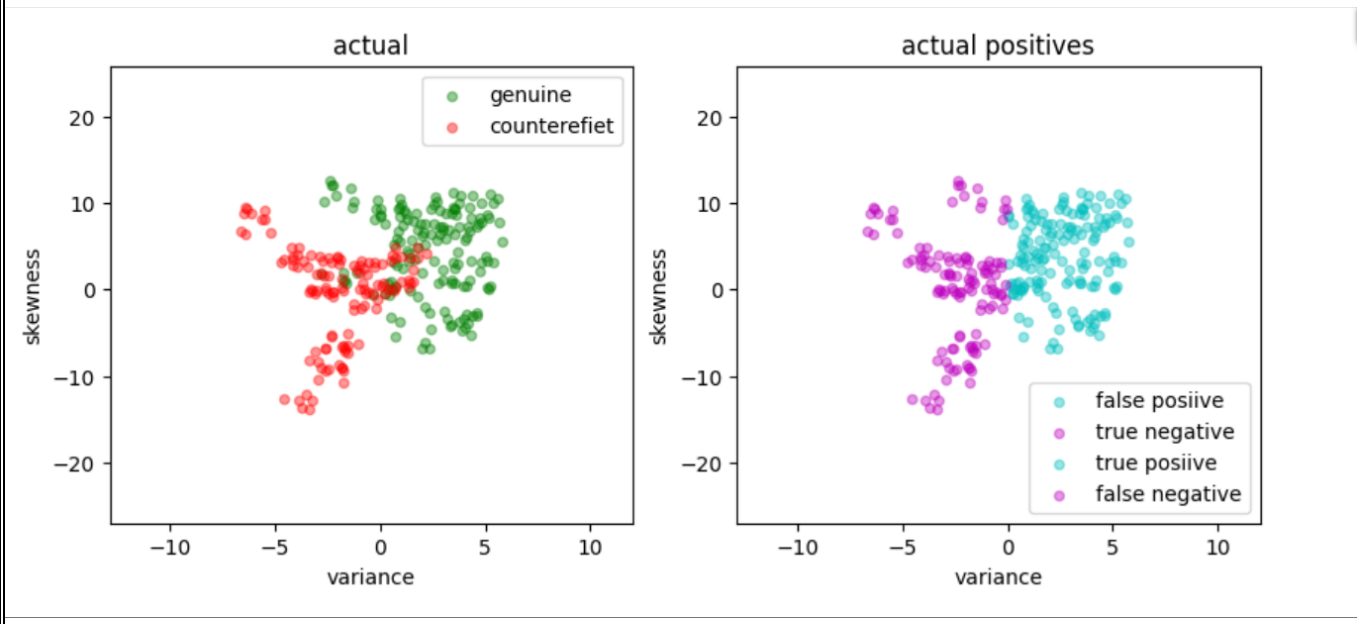
WEEK 10: Write a program to implement SVM for binary classification on a sample training dataset stored as a CSV file with different kernels.

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
df = pd.read_csv(
   "https://raw.githubusercontent.com/ampl/mo-
book.ampl.com/refs/heads/dev/datasets/data_banknote_authentication.txt",
   header=None,)
df.columns=["variance","skewness","curtosis","entropy","class"]
f=df.name= "Banknotes"
df.head()
df.describe()

df_train, df_test = train_test_split(df, test_size=0.2)
```

```
features=["variance","skewness"]
x_train=df_train[features]
y_train=1-2*df_train["class"]
x_test=df_test[features]
y_test=1-2*df_test["class"]

def scatter_labeled_data(x,y,labels=["+1","-1"], colors=["g","r"],**kwargs):
  kw={"x":0, "y":1, "kind": "scatter", "alpha":0.4}
  kw.update(kwargs)
  import warnings
  with warnings.catch_warnings():
    warnings.filterwarnings("ignore")
    kw["ax"] = x[y>0].plot(**kw,c=colors[0], label=labels[0])
    x[y<0].plot(**kw, c=colors[1], label=labels[1])
fig, ax= plt.subplots(1,2, figsize=(10,4))
scatter_labeled_data(
    x_train,
    y_train,
    labels=["genuine", "counterfeit"],
    ax=ax[0],
    title="Training Set",
)
scatter_labeled_data(
    x_test,
    y_test,
    labels=["genuine", "counterfeit"],
    ax=ax[1],
    title= "Test Set",

)


import pandas as pd
import numpy as np
class linearSVM:
  def __init__(self,w,b):
    self.w=pd.Series(w)
    self.b=float(b)
  def __call__(self,X):
    return np.sign(X.dot(self.w)+self.b)
  def __repr__(self):
    return f"linearSVM(w={self.w.to_dict()}, b={self.b})"


w=pd.Series({"variance":1,"skewness":0})
b=0
svm=linearSVM(w,b)
```

```
print(svm)
y_pred=svm(x_test)
accuracy=sum(y_pred==y_test)/len(y_test)
print(f"Accuracy;{100*accuracy:0.1f}%")

def scatter_comparison(X,y,y_pred):
  xmin,ymin=X.min()
  xmax,ymax=X.max()
  xlim=[xmin-0.5*(xmax-xmin),xmax+0.5*(xmax-xmin)]
  ylim=[ymin-0.5*(ymax-ymin),ymax+0.5*(ymax-ymin)]
  labels=["genuine","counterefiet"]
  fig,ax= plt.subplots(1,2, figsize=(10,4))
  scatter_labeled_data(
     X,y, labels,["g","r"],
     ax=ax[0],
     xlim=xlim,
     ylim=ylim,
     title="actual")
  scatter_labeled_data(
     X[y<0],
     y_pred[y<0],
     ["false posiive","true negative"],
     ["c","m"],
     xlim=xlim,
     ylim=ylim,
     ax=ax[1],
     title="actual negatives")
  scatter_labeled_data(
     X[y>0],
     y_pred[y>0],
     ["true posiive","false negative"],
     ["c","m"],
     xlim=xlim,
     ylim=ylim,
     ax=ax[1],
     title="actual positives")

scatter_comparison(x_test,y_test,y_pred)


scatter_comparison(x_test,y_test,y_pred)
```

Output:



Case-1:

```
from sklearn import svm
svm_model = svm.SVC(C=2.0, kernel="linear")
svm_model.fit(x_train,y_train)
ypred= svm_model.predict(x_test)
accuracy=np.mean(ypred==y_test)
print(f"Accuracy: {100*accuracy:0.1f}%")
```

output: Accuracy: 86.2%

Case-2:

```
from sklearn import svm
svm_model = svm.SVC(C=2.0, kernel="rbf")
svm_model.fit(x_train,y_train)
ypred= svm_model.predict(x_test)
accuracy=np.mean(ypred==y_test)
print(f"Accuracy: {100*accuracy:0.1f}%")
```

output: Accuracy: 89.8%

Case-3:

```
from sklearn import svm
svm_model = svm.SVC(C=2.0, kernel="sigmoid")
```

```
svm_model.fit(x_train,y_train)
ypred= svm_model.predict(x_test)
accuracy=np.mean(ypred==y_test)
print(f"Accuracy: {100*accuracy:0.1f}%")
```
Output: Accuracy: 61.5%

Case-4:

```
from sklearn import svm
svm_model = svm.SVC(C=2.0, kernel="poly")
svm_model.fit(x_train,y_train)
ypred= svm_model.predict(x_test)
accuracy=np.mean(ypred==y_test)
print(f"Accuracy: {100*accuracy:0.1f}%")
```
Output: Accuracy: 87.3%

```
def decision_boundary_plot(X,y,model):
  x_min,x_max=X.iloc[:,0].min()-0.5,X.iloc[:,0].max()+0.5
  y_min,y_max=X.iloc[:,1].min()-0.5,X.iloc[:,1].max()+0.5
  xx,yy=np.meshgrid(np.arange(x_min,x_max,0.02),np.arange(y_min,y_max,0.02))
  x_in=np.c_[xx.ravel(),yy.ravel()]
  y_pred=model.predict(x_in)
  y_pred=y_pred.reshape(xx.shape)
  plt.contourf(xx,yy,y_pred, alpha=.5,cmap='winter')
  plt.scatter(X.iloc[:,0],X.iloc[:,1],c=y,cmap='spring',edgecolor='b')
  plt.show()
decision_boundary_plot(x_test,y_test,svc)


kernels=["linear","sigmoid","poly"]
for i, kernel_name in enumerate(kernels):
  svc = svm.SVC(C=2.0, kernel=kernel_name)
  svc.fit(x_train, y_train)
  decision_boundary_plot(x_test, y_test, svc)
  plt.show()
```

Outputs: